Triangle Rasterization

Introduction

The purpose of this document is to understand the basic mathematical elements and algorithms required to partially replicate in software the behavior of fixed-function rasterization stage in GPUs. The basic geometric primitives defined in OpenGL are points, lines, and triangles. Even though points and lines are mathematically simpler than triangles, it is the case that most $3\mathrm{D}$ engines treat triangles as native rendering primitives. The problem with a mathematical point is that it has no extent while a line has no width. Therefore, OpenGL defines points and lines as mixed $2\mathrm{D}/3\mathrm{D}$ entities: the point's location or line's end points are defined in $3\mathrm{D}$, but after projection, they're given a point size or line width defined in pixels, making them partially $2\mathrm{D}$ entities. Support for quadrilaterals and polygons has been stripped from core OpenGL. Therefore, the particular primitive of interest is the triangle. The specific questions examined in this document are:

1. Why are triangles the de facto primitive in real-time computer graphics?
2. How do GPUs determine the set of fragments inside a triangle primitive?
3. How do GPUs interpolate per-vertex attribute values across the triangle's surface?

To conceptualize the operations performed by programmable and fixed-function stages on triangle primitives, the graphics pipe is reviewed. Understanding the various coordinate systems encountered by vertices and triangles, especially normalized device context (NDC) system and window coordinate system is a necessary criterion for implementing a triangle rasterizer. Note that the graphics community hasn't standardized on the consistent use of the term window coordinate system. Computer graphics texts and literature commonly and interchangeably use the terms screen coordinate system, device coordinate system, viewport coordinate system, and window coordinate system.

The document covers mathematical concepts including implicit line equations, point-normal equations involving dot products, and triangle areas involving cross products. Since triangles share vertices and edges, specific rules must be devised to ensure that only a single triangle is given ownership of a fragment. Given the large number of fragments encompassed by the framebuffer, efficient traversal algorithms must be devised to ensure that a minimal number of fragments are tested for each triangle. The fastest triangle that is rasterized is the triangle that isn't rasterized because it is culled and discarded by the pipeline. The implementation of back-face culling is discussed from the perspective of clockwise and counterclockwise winding of triangle vertices and the signed areas of these triangles. In addition, the particular OpenGL commands and state settings that implement various triangle culling mechanisms are also discussed. The document concludes with a discussion of the use of barycentric coordinates to solve the problem of computing vertex attribute values at interior fragments by interpolating values defined at triangle vertices across the triangle surface.

Review of the OpenGL Graphics Pipeline

The OpenGL graphics pipeline is a sequence of data processing stages where the output of one stage is used as the input to the subsequent stage. Figure 1 shows the OpenGL version 4.6 pipeline.

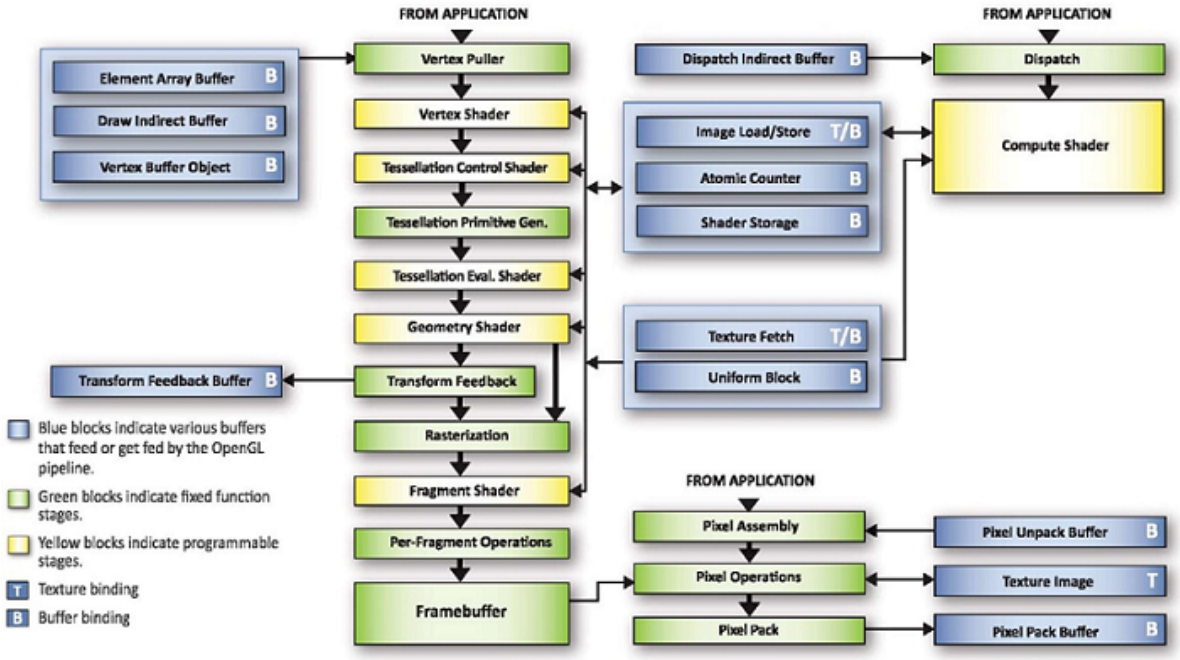

Figure 1: [OpenGL Version 4.6 graphics pipeline]

The OpenGL graphics pipeline accepts as input a geometrical representation of a 3D scene stored in GPU memory. The input data is processed by a variety of shader programs and fixed-function processing stages into a final rendered image. To render a frame using OpenGL, the application configures the graphics pipeline and submits 3D models to be rendered. In general, each model will require a different pipeline configuration based on the model's specific geometric data and topology. 3D models can be represented using a variety of methods including polygonal surfaces, quadric surfaces, spline surfaces, procedural surfaces, physically based modeling methods for hair, fur, cloth, octree encoding, isosurface display, and volume rendering. Irrespective of the representation method, these models must be specified using OpenGL ***primitives*** that will get rendered as one of the three native types supported by all graphics hardware: *points*, *lines* and *triangles*. Line primitive types can be combined together to form *strips* and *loops*; triangle primitive types can be combined together to form *fans* and *strips*.

The main stages in the OpenGL pipeline of concern to us consist of vertex processing, rasterization, fragment processing, and per-fragment operations. Figure 2 uses the path taken by a triangular polygon of a three-dimensional model to illustrate the main processes involved in the generation of the final rendered image.

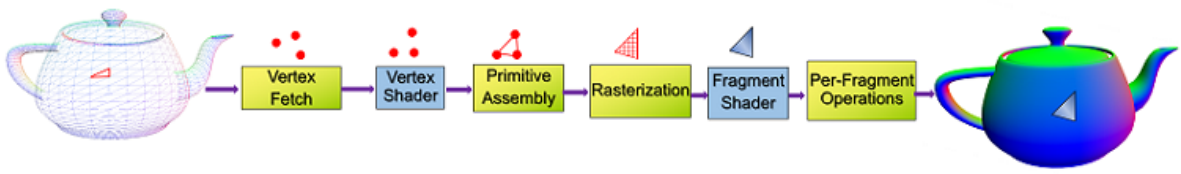

Figure 2: Main stages of simplified OpenGL graphics pipeline

The ***vertex processor*** is a programmable unit that is responsible for transforming incoming geometry into something suitable for consumption by subsequent stages. OpenGL requires geometric data to be presented to the GPU in the form of vertices. A vertex is a bundle of ***vertex attributes*** with each attribute specifying data such as position coordinates, normals, tangents, colors, and texture coordinates. OpenGL requires vertices and the topology that connects vertices into primitives of points, lines, or triangles be stored in ***buffer objects***, which are chunks of memory managed by the GPU. The fixed-function ***vertex fetch*** stage supplies the vertex processor with vertices from buffer storage. The specific buffer memory regions and formats of vertex attributes stored in these buffers is determined by states set by specific OpenGL commands. Compilation units written in the OpenGL Shading Language to be run on the vertex processor are called ***vertex shaders***. The vertex shader is a piece of code *executed once and only once for each vertex*. The vertex processor consists of many cores with each core executing one vertex shader that transforms one vertex allowing computation to be performed on several vertices in parallel, with the parallelization width dependent on the particular graphics hardware being used. Minimally, vertex shaders must transform position coordinates usually specified in model coordinate system and produce corresponding position coordinates in clip coordinate system. Vertex shaders can do more - they can transform and normalize normals, generate and transform texture coordinates, and implement lighting calculations at each vertex. Since vertex shaders are applied on individual vertices, they cannot perform operations such as clipping and culling that require knowledge of several vertices at once.

Using the primitive type specified by either `glDrawArrays` or `glDrawElements`, the fixed-function ***primitive assembly*** stage organizes collections of vertices into a single, complete topological primitive that will then be operated on by fixed-function stages illustrated in Figure 3:

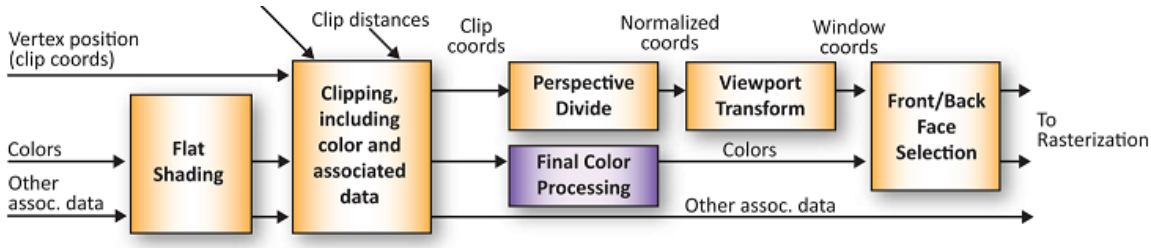

Figure 3: [Official OpenGL follow on stages after vertex shader]

1. ***clipping***: the primitive is clipped against the view frustum in clip coordinate system to eliminate portions that will be projected outside the viewport.
2. ***projection (perspective or orthographic) division***: the vertices of the clipped primitive are projected from clip coordinate system to NDC system.
3. ***viewport transform***: primitives in NDC system are transformed to window (or device or screen) coordinate system using the arguments specified in OpenGL command `glViewport`.
4. ***triangle culling***: triangles are discarded based on whether they're front-facing or back-facing with the interpretation controlled by calls to OpenGL commands `glFrontFace` and `glCullFace`.

Primitives that have not been clipped out nor culled because of their facing orientations are decomposed by fixed-function ***rasterizer*** stage into smaller units called ***fragments*** that correspond to pixels in the framebuffer. Primitives contain a set of attributes at each vertex - for example, the red triangle in Figure 2 will have a set of attributes at each vertex with each set potentially containing texture coordinates, normals for lighting, colors from lighting calculations performed in the vertex shader, depth values computed during projection division, and any other data supplied by the application. The rasterizer will interpolate these per-vertex values across the primitive and assign the interpolated data to each fragment. After rasterization, fragments are processed by early per-fragment tests such as the pixel ownership test, scissor test, and if

enabled the stencil, the depth buffer, and occlusion query sample counting tests are also performed. Think of each fragment generated by the rasterizer as a "may be" pixel because it is possible that the fragment may be rejected by these early per-fragment tests (and other actions performed by the fixed-function per-fragment operations stage) and may never update the corresponding pixel's color value (and if depth buffering is enabled, the corresponding depth buffer).

Surviving fragments are processed by the ***fragment processor*** which is a programmable unit that operates on fragment values. Compilation units written in the OpenGL Shading Language to run on this processor are called ***fragment shaders***. The fragment shader is a piece of code *executed once and only once for each fragment* - the interpolated values contained in a fragment can be used in any computation such as plain color computations, texture mapping, per-pixel lighting, per-pixel fog, blending, and so on. The fragment processor has several cores with each core executing a fragment shader that operates on an individual fragment with the parallelization width depending on the particular graphics hardware being used. The minimum requirement for the fragment shader is to use the interpolated values at each fragment to update the corresponding location in the destination framebuffer's colorbuffer with a color value. The fragment shader may also modify or replace the interpolated depth values of fragments. Changes to a fragment's window position nor access to neighboring fragments is allowed.

After a fragment is modified by the programmable fragment processor to have a color and a depth value, it is then further modified, and possibly discarded by the fixed-function ***per-fragment operations*** stage. Operations performed in this stage are controlled by calls to OpenGL commands and include stencil test, depth buffer test, occlusion query test, blending, dithering, and logic operations. The stencil test, depth buffer test, occlusion query tests can be implemented as early per-fragment tests right after rasterization. Finally, if the fragment was not discarded, its computed color is used to update the destination framebuffer's colorbuffer and its depth value is used to update the destination framebuffer's depthbuffer. The update locations within these buffers is specified by the fragment's window coordinates.

Why Triangles?

Before studying the mathematical elements underlying triangle rasterization, let's take a moment to understand why triangles have become the building blocks for real-time graphics. A few basic definitions are necessary to succinctly describe different varieties of polygons. A ***polyline*** is a finite sequence of line segments with adjacent line segments sharing an end point. A polyline is *closed* if the last point of the line is connected to the first by a line segment. A ***polygon*** is a closed polyline with each end point called a *vertex* of the polygon and each line segment called an *edge* of the polygon. A polygon is said to be ***simple*** if it is non-self-intersecting or if doesn't have any holes. A simple polygon bounds a simply connected region in the plane with points in this region said to be ***inside*** the polygon. ***Complex*** polygons are either self-intersecting or they have holes.

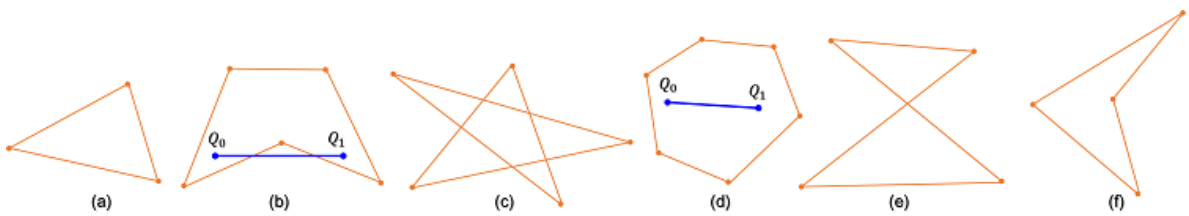

Figure 4: Polygons (a), (b), (d) are *simple* polygons while (c) and (e) are not

Polygons (a), (b), (d), and (f) in Figure 4 are simple polygons while (c) and (e) are not. Among simple polygons, some are ***convex*** in the sense that for any two points in the polygon, the line segment connecting the two points lies completely inside the polygon. Examples of convex polygons include triangles, rectangles, parallelogram, and the six-sided simple polygon (d) in

Figure 4. Polygon (d) is convex since, regardless of how interior points Q_0 and Q_1 are chosen inside the polygon, the line segment connecting them will always be inside the polygon. A simple polygon that is not convex is a ***concave*** polygon in the sense that for any two points in the polygon, the line segment connecting the two points doesn't lie completely inside the polygon. Polygons (b) and (f) are concave since the line segment connecting two interior points Q_0 and Q_1 is not completely inside the polygon.

The triangle is the atomic unit of rendering in real-time computer graphics - in fact, graphics hardware is optimized for processing triangle primitives. There are three main reasons for this.

1. Any three-dimensional shape can be represented using triangles with the "smoothness" of the underlying surface controlled by the density of triangles. There is nothing to be gained by using arbitrary polygons since triangulation algorithms can convert any arbitrary polygon into triangles. Although triangulation algorithms for arbitrary polygons are non-trivial, convex polygons with N vertices can be easily divided into a fan of N-2 triangles. Triangulated shapes offer a number of advantages over general polygon shapes. The data structures that represent a general polygon shape are complicated by the fact that each face can involve an arbitrary number of vertices and edges. This problem doesn't arise for triangulated shapes, since each of its faces is defined by exactly three vertices and three edges. Normal vectors, used extensively in a variety of graphics operations, can easily be computed for triangles. The direction of the normal comes from the cross product of vectors defined by any two of the triangle's edges. This procedure cannot be used for an arbitrary polygon because some of its edges may be collinear. A normal vector computed this way is guaranteed to be correct across the whole surface of the triangle, a fact that is not true for general polygon since they can be warped.
2. Triangles are always convex. Inside-outside tests for an arbitrary point with respect to a surface are extensively used in clipping, collision detection, and rasterization algorithms. These tests are computationally simpler for triangles. Dealing with any other type of polygon other than triangles makes hardware implementations more complex. For example, a scanline can intersect with more than two polygon edges - see polygon (b) in Figure 4. Consider the problems involved in rendering self-intersecting polygons such as the one in Figure $4(\mathrm{c})$ - defining which regions of the $2\mathrm{D}$ plane are inside and which are outside for such polygons is much more difficult compared to the triangle in Figure $4(\mathrm{a})$. It may seem that quadrilaterals - the next simplest polygons compared to triangles - might provide a good balance between triangles and more arbitrary polygons. However, quadrilaterals can be self-intersecting as in Figure $4(\mathrm{e})$ or concave as in Figure $4(\mathrm{f})$.
3. Triangles are always planar. Given three vertices, either the vertices are collinear and will define a degenerate triangle or they will define a flat, $2\mathrm{D}$ surface. What about quadrilaterals? Even if quadrilaterals are not self-intersecting or concave, the four vertices need not be in the same plane making it more difficult to render them in a simple way since there are many different ways to define which points are on the surface of that primitive.
4. Triangles are the only polygon to have the property that every point inside them can be uniquely represented in terms of its vertex position coordinates. This makes it possible for values defined at the vertices to be unambiguously interpolated throughout the primitive. Therefore, the hardware to interpolate triangles is greatly simplified in comparison to other convex and concave polygons.

In conclusion, triangles provide the cleanest and simplest definition of a rendering primitive that fulfils the requirements of representing a piece of a $\text{2D} surface: they are always convex, they always reside in a single plane, and interpolating vertex attributes is easier to define and compute.

Window Coordinates

The purpose of rasterization is to identify fragments that contribute to the final image in the destination framebuffer's colorbuffer. As indicated by the earlier review of the graphics pipe, rasterization is implemented in the window coordinate system - a small portion of this coordinate system is illustrated in Figure 5.

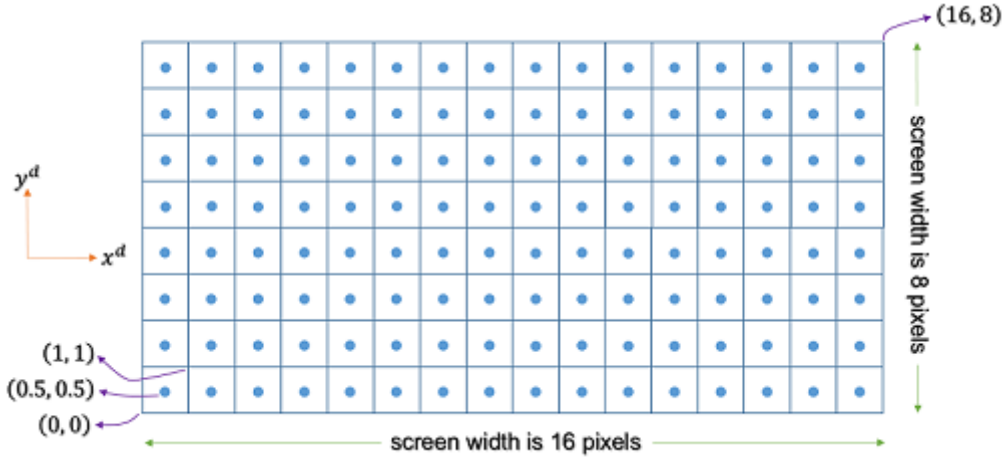

Figure 5: OpenGL's window coordinate system

Rasterizing a primitive requires determining which squares of an integer grid in window coordinates are occupied by the primitive. A grid square, including its (x, y) window coordinates, z (depth), and associated data which may be added by fragment shaders, is called a ***fragment***. A fragment is located by its lower left corner, which lies on integer grid coordinates. Rasterization operations also refer to a ***fragment's center***, which is offset by $(0.5, 0.5)$ from its lower left corner and so lies on half-integer coordinates. Thus, the center of a fragment (x, y) is given by $(x + 0.5, y + 0.5)$, where $x \in [0, w - 1]$ and $y \in [0, h - 1]$ are integers, and $w \times h$ is the window resolution as determined by the OpenGL command `glViewport`, e.g., 2400×1350 or 1600×900 pixels.

Side note: Starting with version 4.5, OpenGL now allows programmers to define the upper left corner as the origin of the window coordinate system through the `glClipControl` command. This feature was introduced to make it easier to port Direct3D applications to OpenGL. This document will make no further mention of this command and will always assume the lower left corner as the origin of the window coordinate system.

Edge Equations

In this section, triangle edge equations - fundamental to rasterizing triangles in hardware - are explained. Figure 6 shows a two-dimensional triangle described by three counter-clockwise oriented vertices, P_0^d, P_1^d, and P_2^d, where $P_i^d = (x_i^d, y_i^d)$. The superscript d indicates that these vertices are described in the window coordinate system. After these vertices are transformed by the vertex shader and several fixed-function stages, they'll have depth values and various other parameters. These values are not required in this portion of the rasterizer and are so omitted. Since mathematical concepts are being described, the terms vertices and points will be used interchangeably in the following discussion.

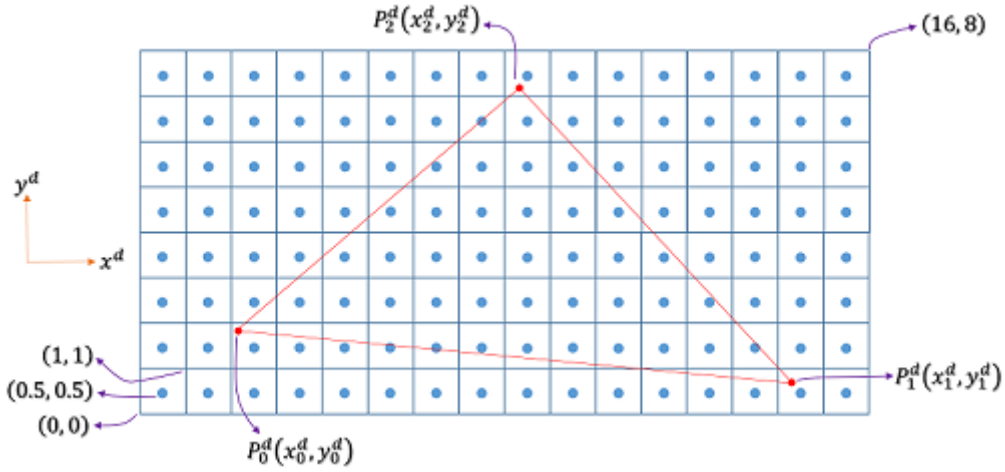

Figure 6: Triangle in window coordinates with counter-clockwise winding

Consider, as shown in Figure 7, a directed line segment from point $P_0^d = (x_0^d, y_0^d)$ to point $P_1^d = (x_1^d, y_1^d)$. The vector from P_0^d to P_1^d and the (infinite) line passing through these two points can be used to divide the entire two-dimensional coordinate system into three regions: all points to the "*left*" or "*inside*" of, to the "*right*" or "*outside*" of, and exactly "*on*" the line. Make sure to understand that the notion of *left*/*inside* and *right*/*outside* is determined by the direction of the vector from the edge's start point to end point. The *inside* and *outside* regions are flipped when a directed line segment from ${P_1}^d$ to P_0^d is used.

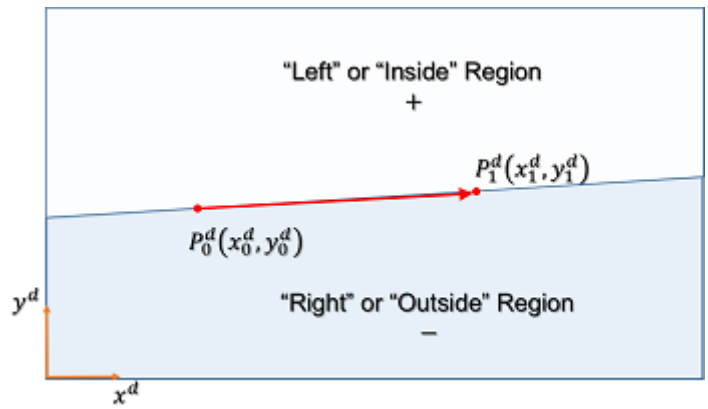

Figure 7: "Inside" and "Outside" half-planes of directed edge

Implicit Edge Equations

An edge equation of a line is an implicit equation, $ax + by + c = 0$, where a, b, and c are some constants determined by two points on the line. First, the vector from start point P_0^d to end point P_1^d is computed: $\vec{e} = P_1^d - P_0^d = (x_1^d - x_0^d, y_1^d - y_0^d)$. Next, this edge vector $\vec{e}$ is rotated by an angle of 90° about vertical z-axis to obtain a normal vector $\vec{n}$ orthogonal to $\vec{e}$ and oriented from *outside* region to *inside* region:

$$\begin{aligned}
\vec{n} = \begin{bmatrix} n_x \\ n_y \end{bmatrix} &= \begin{bmatrix} \cos(90^\circ) & -\sin(90^\circ) \\ \sin(90^\circ) & \cos(90^\circ) \end{bmatrix} \begin{bmatrix} x_1^d - x_0^d \\ y_1^d - y_0^d \end{bmatrix} \\
&= \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} x_1^d - x_0^d \\ y_1^d - y_0^d \end{bmatrix} \\
\implies \vec{n} = \begin{bmatrix} n_x \\ n_y \end{bmatrix} &= \begin{bmatrix} -(y_1^d - y_0^d) \\ x_1^d - x_0^d \end{bmatrix} = \begin{bmatrix} y_0^d - y_1^d \\ x_1^d - x_0^d \end{bmatrix}
\end{aligned} \tag{1}$$

Next, the dot product between normal vector $\vec{n}$ and vector from P_0^d to an arbitrary point $P^d = (x^d, y^d)$ is computed:

$$
\begin{aligned}
\vec{n} \bullet (P^d - P_0^d) &= \begin{bmatrix} y_0^d - y_1^d \\ x_1^d - x_0^d \end{bmatrix} \bullet \begin{bmatrix} x^d - x_0^d \\ y^d - y_0^d \end{bmatrix} \\
\implies \vec{n} \bullet (P^d - P_0^d) &= (y_0^d - y_1^d)(x^d - x_0^d) + (x_1^d - x_0^d)(y^d - y_0^d) \qquad (2)
\end{aligned}
$$

The edge function for the line defined by the directed edge from start point P_0^d to the end point P_1^d is now described as:

$$
\begin{aligned}
E(x^d, y^d) &= (y_0^d - y_1^d)(x^d - x_0^d) + (x_1^d - x_0^d)(y^d - y_0^d) = ax^d + by^d + c, \text{where} \qquad (3) \\
a &= y_0^d - y_1^d, \\
b &= x_1^d - x_0^d, \\
c &= -(y_0^d - y_1^d)x_0^d - (x_1^d - x_0^d)y_0^d = x_0^d y_1^d - x_1^d y_0^d
\end{aligned}
$$

Point-Normal Edge Equations

To obtain a geometric interpretation of implicit equation (3) and relate it to Figure 7, the equation is rewritten as a point-normal equation

$$
\begin{aligned}
E(x^d, y^d) &= ax^d + by^d + c \\
&= \vec{n} \bullet (P^d - P_0^d) \\
&= \vec{n} \bullet P^d - \vec{n} \bullet P_0^d \\
&= \begin{bmatrix} n_x \\ n_y \end{bmatrix} \bullet \begin{bmatrix} x^d \\ y^d \end{bmatrix} - \begin{bmatrix} n_x \\ n_y \end{bmatrix} \bullet \begin{bmatrix} x_0^d \\ y_0^d \end{bmatrix} \\
\implies E(x^d, y^d) &= \begin{bmatrix} y_0^d - y_1^d \\ x_1^d - x_0^d \end{bmatrix} \bullet \begin{bmatrix} x^d \\ y^d \end{bmatrix} - \begin{bmatrix} y_0^d - y_1^d \\ x_1^d - x_0^d \end{bmatrix} \bullet \begin{bmatrix} x_0^d \\ y_0^d \end{bmatrix} \qquad (4)
\end{aligned}
$$

For all points $P^d = (x^d, y^d)$ located exactly on the infinite extension of the edge from P_0^d to P_1^d, Equation (4) holds that $E(x^d, y^d) = 0$. For points in the "inside" region, Equation (4) evaluates to $E(x^d, y^d) > 0$, and for points in the "outside" region, Equation (4) evaluates to $E(x^d, y^d) < 0$. Again, it is important for the reader to understand that the edge equation can be defined so that the opposite holds as well. This convention has been chosen since it is the most convenient for working with triangles that have counter-clockwise winding.

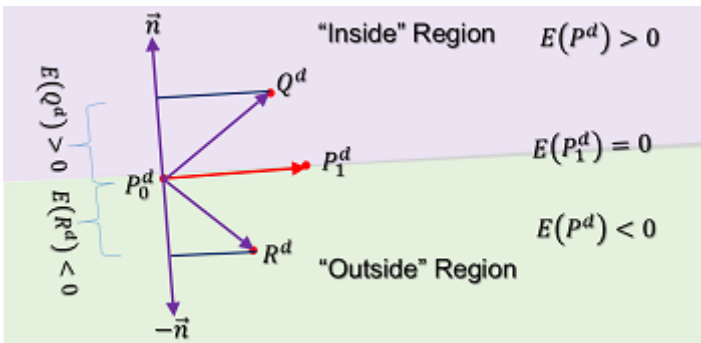

Figure 8: Geometrical relationship between point in plane and edge function

Let's build some intuition on why the edge functions work like this. Figure 8 illustrates an example of an edge function where normal vector $\vec{n} = (y_0^d - y_1^d, x_1^d - x_0^d)$ to the edge from P_0^d to P_1^d has the same length as edge vector $\vec{e} = P_1^d - P_0^d$ but is orthogonal to it. Mathematically, this means that $\vec{n} \bullet (P_1^d - P_0^d) = 0$. That is, when a point P_1^d on the edge (or its infinite extension) is plugged into Equation (4), 0 is returned. When point Q^d is plugged into Equation (4), the projection of vector $Q^d - P_0^d$ onto normal vector $\vec{n}$ is returned: $E(Q^d) = \vec{n} \bullet (Q^d - P_0^d)$. If point Q^d is on the same side of the edge as normal $\vec{n}$, the projection must be positive. This intuition is borne out by Figure 8 which shows that point Q^d is in the "inside" region with respect to the edge. Similarly, when the point R^d is plugged into Equation (4), vector $R^d - P_0^d$ is projected onto normal vector $\vec{n}$ giving a negative result if point R^d is on the side of the edge where the normal is

not oriented. This intuition is borne out by Figure 8 which shows that point R^d is in the "outside" region with respect to the edge.

The intuition works in both directions. That is, if an arbitrary point P^d is plugged into Equation (4) and $E(P^d)$ evaluates to 0, then point P^d *must* be on the infinite extension of the edge; if $E(P^d)$ evaluates to a positive result, then point P^d *must* be located in inside region of the edge; and if $E(P^d)$ evaluates to a negative result, then point P^d *must* be located in outside region of the edge. These results as gathered as:

$$\begin{aligned} &E(P^d) > 0 \Leftrightarrow P^d \text{ is in } \textit{inside} \text{ region of edge,} \\ &E(P^d) < 0 \Leftrightarrow P^d \text{ is in } \textit{outside} \text{ region of edge,} \\ &E(P^d) = 0 \Leftrightarrow P^d \text{ is } \textit{on} \text{ edge} \end{aligned} \tag{5}$$

For each triangle (see Figure 6), three edge equations are generated with edge equation $E_i(P^d)$ derived from vertices opposite to vertex P_i^d. That is, edge equation E_0 is the directed edge from vertex P_1^d to vertex P_2^d; edge equation E_1 is the directed edge from vertex P_2^d to vertex P_0^d; and edge equation E_2 is the directed edge from vertex P_0^d to vertex P_1^d. The three equations are listed below:

$$\begin{aligned} E_0(P^d = (x^d, y^d)) &= (y_1^d - y_2^d)(x^d - x_1^d) + (x_2^d - x_1^d)(y^d - y_1^d) = a_0 x^d + b_0 y^d + c_0, \\ E_1(P^d = (x^d, y^d)) &= (y_2^d - y_0^d)(x^d - x_2^d) + (x_0^d - x_2^d)(y^d - y_2^d) = a_1 x^d + b_1 y^d + c_1, \\ E_2(P^d = (x^d, y^d)) &= (y_0^d - y_1^d)(x^d - x_0^d) + (x_1^d - x_0^d)(y^d - y_0^d) = a_2 x^d + b_2 y^d + c_2 \end{aligned} \tag{6}$$

Edge Equations and Signed Triangle Areas

So far, edge equation $E(P^d)$ has been studied from the perspective of implicit line equations: $E\left(P^d = \left(x^d, y^d\right)\right) = ax^d + by^d + c$ (see Equations (3) and (6)) and from the perspective of dot products in the form of point-normal equations: $E\left(P^d = \left(x^d, y^d\right)\right) = \vec{n} \bullet \left(P^d - P_{start}^d\right)$ (see Equations (4) and (6)). A third method to view edge equations comes from the intuition of using the cross product operation to compute a triangle's area. Rearranging terms in Equation (6) for edge equation $E_2\left(P^d\right)$ (the edge from P_0^d to P_1^d), we have:

$$\begin{aligned} E_2\left(P^d = \left(x^d, y^d\right)\right) &= -\left(y_1^d - y_0^d\right)\left(x^d - x_0^d\right) + \left(x_1^d - x_0^d\right)\left(y^d - y_0^d\right) \\ &= \left(x_1^d - x_0^d\right)\left(y^d - y_0^d\right) - \left(y_1^d - y_0^d\right)\left(x^d - x_0^d\right) \\ &= \begin{vmatrix} x_1^d - x_0^d & x^d - x_0^d \\ y_1^d - y_0^d & y^d - y_0^d \end{vmatrix} \\ &= \left(\left(P_1^d - P_0^d\right) \times \left(P^d - P_0^d\right)\right) \bullet (0, 0, 1) \\ &= 2 \times \text{signed area of } \Delta P_0^d P_1^d P^d \end{aligned} \tag{7}$$

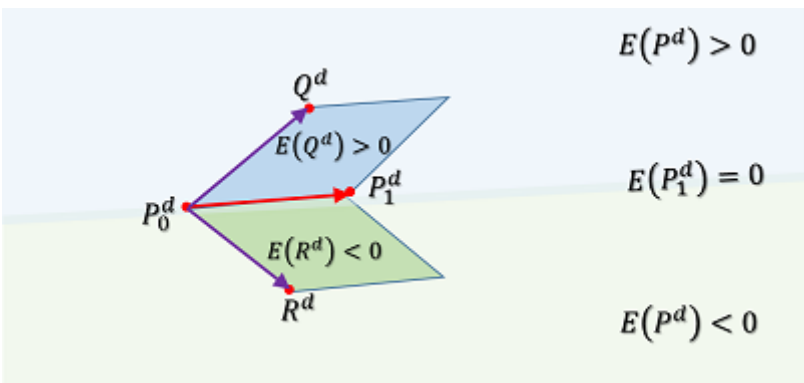

Figure 9: Inside/Outside tests defined by signed triangle areas

Let's look at the intuition between triangle areas, insideness, and outsideness of points with respect to edges. Figure 9 illustrates the scalar triple product $((P_1^d - P_0^d) \times (Q^d - P_0^d)) \bullet (0,0,1)$ computing the positive area of the blue parallelogram for a point Q^d in the inside region of edge from P_0^d to P_1^d. The computed area is positive because $\Delta P_0^d P_1^d Q^d$ describes triangles with counter-clockwise orientation for all points Q^d in the inside region of edge from P_0^d to P_1^d. The cross product between vectors $P_1^d - P_0^d$ and $Q^d - P_0^d$ is oriented in a direction orthogonal to the two-dimensional plane - the two alternative directions are either $(0,0,1)$ or $(0,0,-1)$. This document uses the convention that the window coordinate system is left-handed, that is, z^d axis is oriented away from the reader. Using the left-hand rule on the vectors illustrated in Figure 9, the cross product $(P_1^d - P_0^d) \times (Q^d - P_0^d)$ is oriented in direction $(0,0,1)$ which is the direction of $+z^d$ axis of left-handed $3\mathrm{D}$ window coordinate system. And, therefore, the signed area of the parallelogram spanned by vectors $P_1^d - P_0^d$ and $Q^d - P_0^d$ is positive.

On the other hand, the green parallelogram's area is negative because $\Delta P_0^d P_1^d R^d$ describes triangles with clockwise orientation for all points R^d in the outside region of edge from P_0^d to P_1^d. Using the left-hand rule on the vectors illustrated in Figure 9, the cross product $(P_1^d - P_0^d) \times (R^d - P_0^d)$ is oriented in direction $(0,0,-1)$ - the direction of $-z^d$ axis of left-handed $3\mathrm{D}$ window coordinate system. And, therefore, the signed area of the parallelogram spanned by vectors $P_1^d - P_0^d$ and $R^d - P_0^d$ is negative.

Implementation of Edge Equations

The following pseudocode takes a directed edge defined by a start point and an end point and returns the constants a, b, and c in the edge equation $E(x^d, y^d) = ax^d + by^d + c$:

```
EdgeEqn(start point P0, end point P1) {
  a = P0.y - P1.y
  b = P1.x - P0.x
  c = P0.x*P1.y - P1.x*P0.y
  return a, b, c
}
```

Next, using function `EdgeEqn`, three edge equations for a triangle are computed:

```
TriangleEdgeEqn(counterclockwise oriented vertices P0, P1, P2) {
  E0 = EdgeEqn(P1, P2)
  E1 = EdgeEqn(P2, P0)
  E2 = EdgeEqn(P0, P1)
  return E0, E1, E2
}
```

A function called `PointInEdge` can be written to determine whether a point is inside or outside a directed edge - if the point is inside the edge, the function returns `true`, otherwise `false`:

```
PointInEdge(edge equation E, arbitrary point P) {
  if (E.a*P.x + E.b*P.y + E.c) >= 0 return true
  return false
}
```

Point inside Triangle

Using the three equations in (6), an arbitrary point $P^d = (x^d, y^d)$ can be determined to be inside the triangle if $E_i(P^d) \geq 0 \; \forall \; i \in [0, 1, 2]$. This rule is illustrated in Figure 10 (the lengths of normals have arbitrary scale in the figure and are drawn this way to make the figure compact).

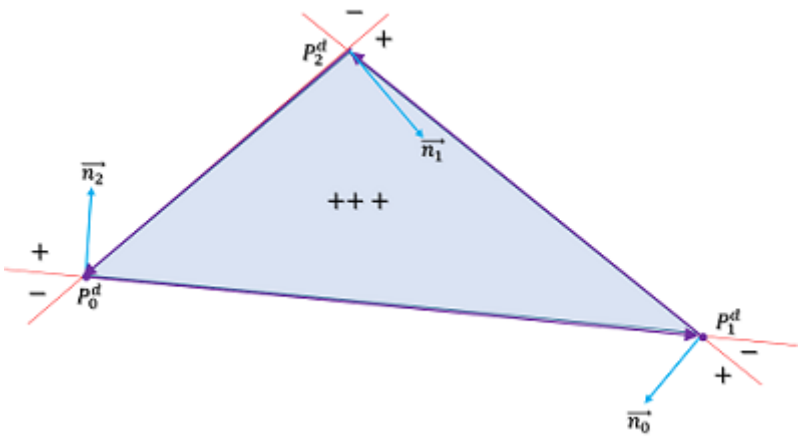

Figure 10: Triangle interior is intersection of inside half-planes of three edges

A function called `PointInTriangle` can be written to implement the point-in-triangle test with the function returning `true` if an arbitrary point is inside the triangle, otherwise returning `false`:

```
PointInTriangle(three edge equations E0, E1, and E2, arbitrary point P) {
  if (PointInEdge(E0, P) && PointInEdge(E1, P) && PointInEdge(E2, P)) return true
  return false
}
```

Identifying Front- and Back-faces

Most opaque, solid $3D$ models provide volume by creating closed surfaces that are tessellated as a mesh of triangles without the need to model the inner surfaces or interior volume. In a $3D$ scene, the front portion of such an opaque, solid model will hide the back portion from the direct line of sight of the viewer (see Figure 11). That is, the model itself occludes certain portions of the model from the viewer. What is invisible need not be rendered since it doesn't contribute to the image. Culling primitives that define backs of objects can conservatively eliminate about half the geometry in the scene from being rendered. The process of removing back facing triangles and not rasterizing them at all is called ***back-face culling***. Back-face culling may not be appropriate in certain scenarios such as when viewing translucent surfaces, non-closed models, or when the image is affected by back-faces that are visible in shadows and reflections.

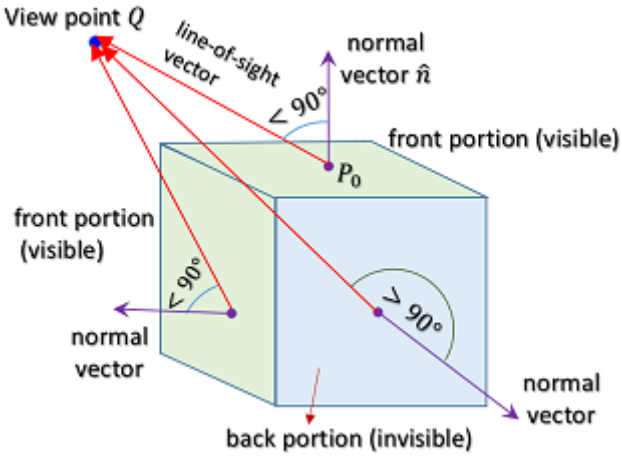

Figure 11: Green portion of model is front-facing while blue portion is back-facing with respect to viewer.

Although the back portion of a model is intuitively recognized - it is what is not visible - how is it identified geometrically? The geometrical tests for identifying front- and back-faces can be written as computing the angle between the face's geometric normal and the line-of-sight vector from a point on the face to view point Q (see Figure 11). If the angle is less than 90° and thus the cosine of this angle is positive, then the face is visible to the viewer. Otherwise, if the angle is greater than 90° and thus the cosine of this angle is negative, then the face is invisible to the viewer. If the angle is exactly 90°, the face is viewed edge on and will be projected as a line and in this case too, the face is assumed to be invisible since the projected line will be rendered by an adjacent face:

$$\begin{aligned} \hat{n} \bullet (Q - P_0) > 0 &: \text{surace is a front-face} \\ \hat{n} \bullet (Q - P_0) \leq 0 &: \text{surface is a back-face} \end{aligned} \tag{8}$$

A second way to understand back-face culling is that just as 2D edges divide the xy plane into inside and outside half-plane regions, a 3D surface such as a triangle divides the 3D xyz space into inside and outside half-spaces. However, in 3D, it is more convenient to use a reversed notion of inside and outside regions of edges with respect to a plane. Consider a closed triangular mesh that is modeled so that triangle normals are oriented from the model's interior to the exterior (this is typically how modeling software generate normals). If P_0 is a point on the triangle and has ***outward normal*** $\hat{n}$, the triangle defines a plane $\mathcal{L} : \hat{n} \bullet (P - P_0) = 0$ where P is an arbitrary point on the plane. Plane $\mathcal{L}$ divides 3D space into an outside (or positive) half-space and inside (or negative) half-space. Computing $\hat{n}$ as an outward normal means $\hat{n}$ is oriented from inside half-space to outside half-space. Figure 12 illustrates this geometrical relationship between a triangle's plane equation and the viewer's position. Let the viewer be located at location Q outside the surface. The triangle is a ***front-face*** with respect to Q if Q lies in the outside half-space of the triangle and the triangle is a ***back-face*** if Q lies on the plane or in the inside half-space of the triangle.

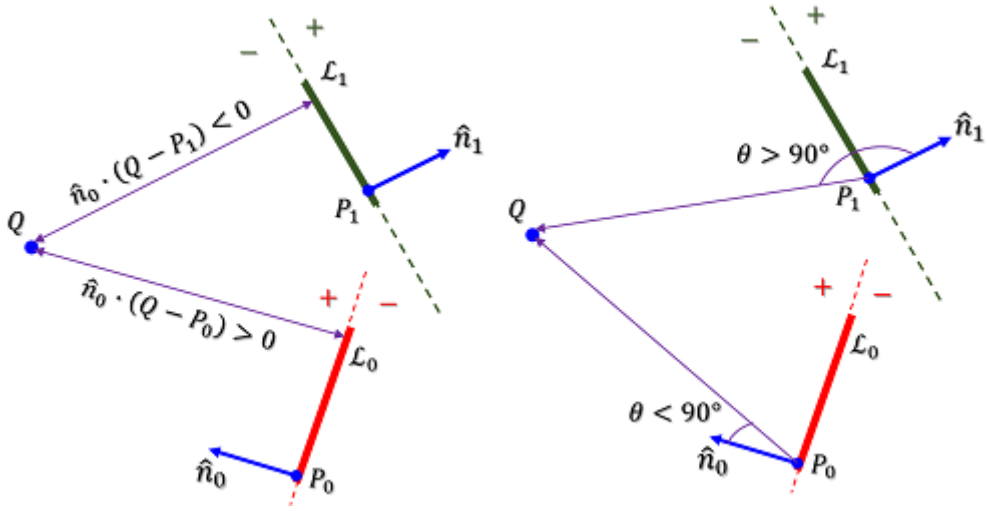

Figure 12: A surface divides 3D space into *inside* (-) and *outside* (+) regions. Left: Test whether the view point is in inside or outside half-plane of triangle's plane. Right: Measure angle between triangle's normal vector and line-of-sight vector from point on triangle to viewer.

To compute an outward oriented normal (from the inside half-space to outside half-space), a winding convention must be specified. The general convention in the computer graphics community is to define triangles so that when viewed from the outside, the vertices are defined using *counterclockwise winding*. Given this convention, the normalized cross product of vectors $\vec{u} = P_1 - P_0$ and $\vec{v} = P_2 - P_0$ gives the surface normal $\hat{n}$ of $\Delta P_0 P_1 P_2$ as shown in Figure 13.

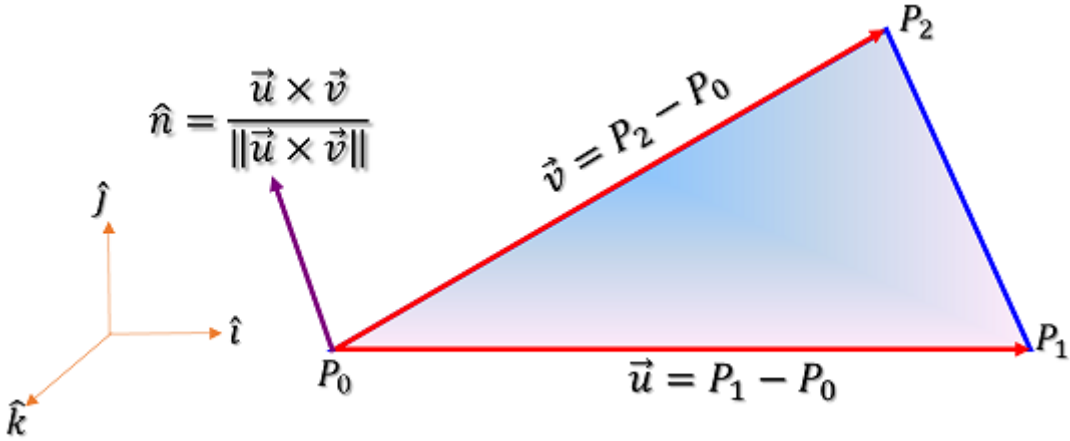

Figure 13: Calculating the normal vector to a triangle with counterclockwise winding.

Notice the use of counterclockwise winding for triangle vertices and the right hand rule in determining the orientation of the normal - the normal points away from the triangle and not into the model of which the triangle is a part of.

If $\hat{i}$, $\hat{j}$, and $\hat{k}$ are the basis vectors, that is, the directions of the x, y, and z axes, then the cross product of vectors $\vec{u} = P_1 - P_0 = (u_{\hat{i}}, u_{\hat{j}}, u_{\hat{k}})$ and $\vec{v} = P_2 - P_0 = (v_{\hat{i}}, v_{\hat{j}}, v_{\hat{k}})$ is computed as:

$$\vec{N} = \vec{u} \times \vec{v} = \begin{vmatrix} \hat{i} & \hat{j} & \hat{k} \\ u_{\hat{i}} & u_{\hat{j}} & u_{\hat{k}} \\ v_{\hat{i}} & v_{\hat{j}} & v_{\hat{k}} \end{vmatrix} = \hat{i} \begin{vmatrix} u_{\hat{j}} & u_{\hat{k}} \\ v_{\hat{j}} & v_{\hat{k}} \end{vmatrix} - \hat{j} \begin{vmatrix} u_{\hat{i}} & u_{\hat{k}} \\ v_{\hat{i}} & v_{\hat{k}} \end{vmatrix} + \hat{k} \begin{vmatrix} u_{\hat{i}} & u_{\hat{j}} \\ v_{\hat{i}} & v_{\hat{j}} \end{vmatrix}$$

$$\implies \vec{N} = \begin{vmatrix} u_{\hat{j}} v_{\hat{k}} - u_{\hat{k}} v_{\hat{j}} \\ u_{\hat{k}} v_{\hat{i}} - u_{\hat{i}} v_{\hat{k}} \\ u_{\hat{i}} v_{\hat{j}} - u_{\hat{j}} v_{\hat{i}} \end{vmatrix} \tag{9}$$

$$\hat{n} = \frac{(N_{\hat{i}}, N_{\hat{j}}, N_{\hat{k}})}{\sqrt{N_{\hat{i}}^2 + N_{\hat{j}}^2 + N_{\hat{k}}^2}}$$

Graphics hardware doesn't perform back-face culling in a $3D$ coordinate system. Instead, as illustrated in OpenGL's graphics pipe and specification (see Figure 3), back-face culling is implemented using window coordinates after primitive assembly, clipping, projection, and viewport transform but before rasterization. Figure 14 shows that things become much simpler in window coordinates. A front-face triangle with counterclockwise winding from camera's point of view will continue to have counterclockwise winding after projection to the window. Similarly, a back-face triangle with clockwise winding from the camera's point of view (but still specified with counterclockwise winding within model geometry) will continue to have clockwise winding after projection. Hence, back-face culling of a triangle only requires determining the vertex winding of the corresponding $2D$ triangle projected into window coordinate system.

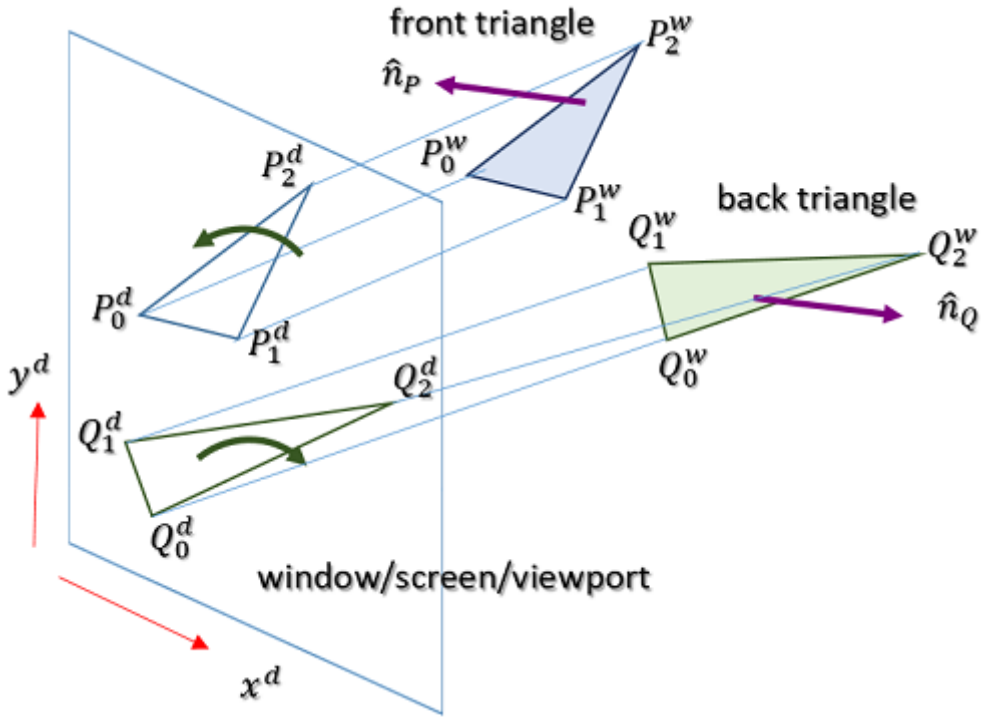

Figure 14: Assuming that all triangles of 3D model have counterclockwise winding, in window coordinates front-facing 2D triangles have counterclockwise winding, while back-facing 2D triangles have clockwise winding.

Using Equation (7) and Figure (9), it is straightforward to determine whether a $2D$ triangle defined in the window coordinate system has clockwise or counterclockwise winding by computing their signed areas. Given $\Delta P_0^d P_1^d P_2^d$ where $P_i^d = (x_i^d, y_i^d)$, define a vector from P_0^d to P_1^d as $P_1^d - P_0^d = (x_1^d - x_0^d, y_1^d - y_0^d)$ and a vector from P_0^d to P_2^d as $P_2^d - P_0^d = (x_2^d - x_0^d, y_2^d - y_0^d)$. Next, compute the following determinant:

$$\text{Signed area of } \Delta P_0^d P_1^d P_2^d = \begin{vmatrix} x_1^d - x_0^d & x_2^d - x_0^d \\ y_1^d - y_0^d & y_2^d - y_0^d \end{vmatrix} = (x_1^d - x_0^d)(y_2^d - y_0^d) - (x_2^d - x_0^d)(y_1^d - y_0^d) \quad (10)$$

If Equation (10) evaluates to a positive value, the triangle has counterclockwise winding. If negative, the winding is clockwise. If zero, the triangle is an edge on face. The resemblance to Equation (7) comes from the fact that $\Delta P_0^d P_1^d P_2^d$ has counterclockwise winding if P_2^d is in the inside region of the edge from P_0^d to P_1^d and has clockwise winding if P_2^d is in the outside of the edge. This can be confirmed from Figure 15 which shows $\Delta P_0^d P_1^d P_2^d$ with counterclockwise winding since vertex P_2^d is in the inside region of edge from P_0^d to P_1^d and $\Delta Q_0^d Q_1^d Q_2^d$ has clockwise winding because Q_2^d is in the outside region of edge from Q_0^d to Q_1^d.

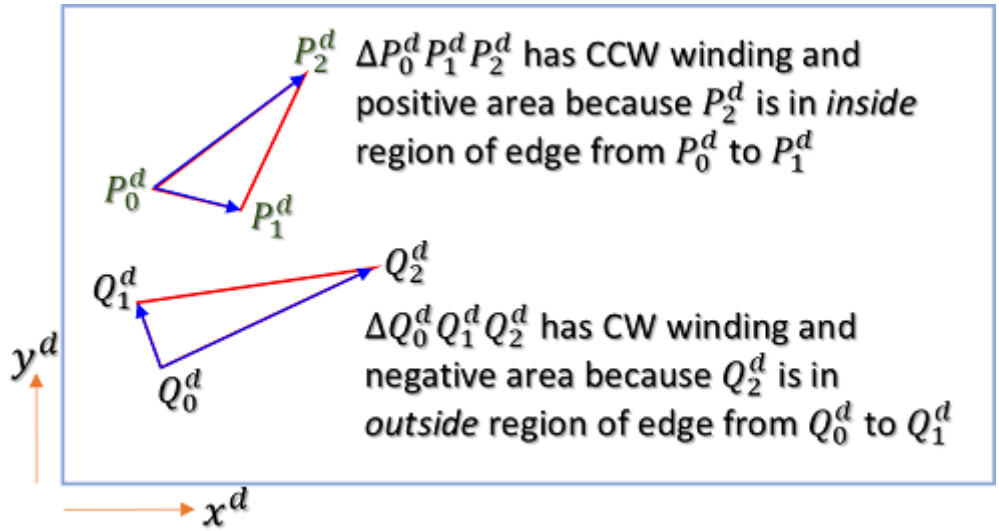

Figure 15: 2D triangles with counterclockwise winding have positive areas while triangles with clockwise winding have negative areas.

Rasterizing Triangles with Edge Equations

Rasterizing triangles involves the following steps:

1. Computing edge equations from vertices.
2. Computing interpolation equations from vertex attributes.
3. Implementing a triangle traversal algorithm to identify fragments inside triangle by evaluating edge equations on squares of the integer grid defined in window coordinate system.
4. Interpolate vertex attributes at these inside fragments.

The computation of edge equations has been discussed in the previous section. The interpolation of vertex attributes will be discussed in a separate document. This section will describe how the integer grid is traversed and specific fragments are chosen.

Point Sampling

The process of converting a continuous function into a series of discrete values is called *sampling*. In essence, triangle rasterization is the process of taking an infinite number of infinitely small mathematical points comprising the triangle interior and mapping them to a finite number of fragments having finite area coverage. The triangle rasterizer will use the ***point sampling*** rule for determining which fragments are produced by the triangle rasterizer. In point sampling, only the center of the fragment will be used when testing whether a fragment is considered to belong to a triangle. Figure 16 illustrates point sampling - only fragment centers that lie inside the triangle are produced by rasterization.

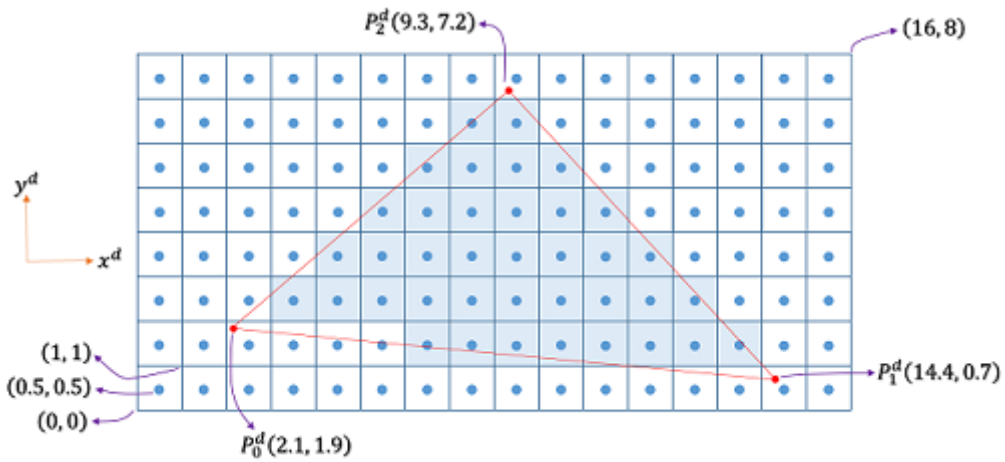

Figure 16: Point sampling applied on triangle in window coordinate system

Point sampling results in unwanted artifacts. For example, consider a scene composed of some black and white triangles. When point sampling the image, near the boundary between a white and black triangle, the image will display some sort of staircase pattern; this artifact is referred to as "the jaggies" (see left image in Figure 17).

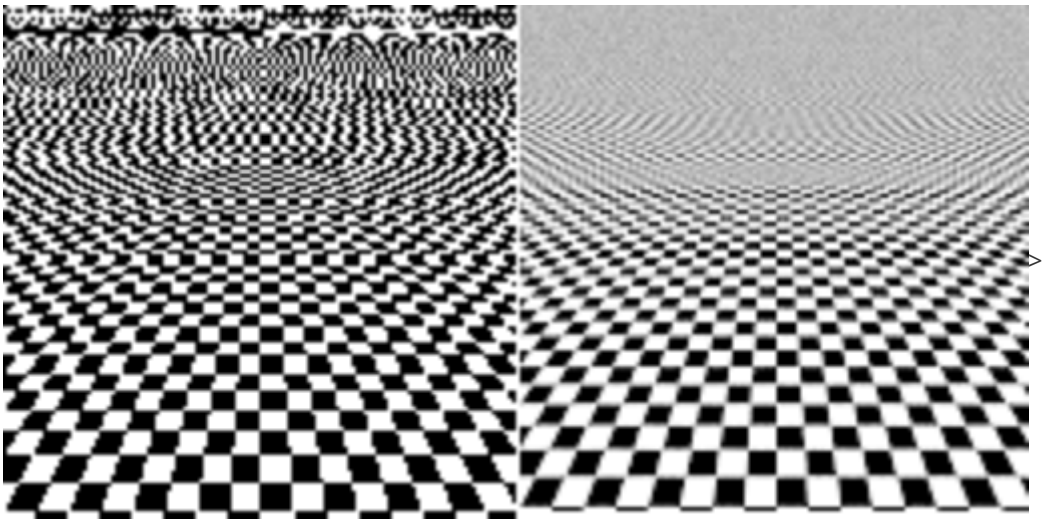

Figure 17: Left: Jaggies at the edge of squares of a checkerboard drawn as a bunch of little black and white squares. Right: Anti-aliased version. (image sourced from here)

During animation, some pixel values will abruptly change from white to black, and the jagged edge pattern will appear to crawl. This is called "the crawling jaggies." Other bad artifacts can arise when dealing with very small triangles. Suppose the checkerboard pattern is far away in the background, so far away that its projection covers only a single fragment. What will be the color of this fragment? If the point sample happens to land on a black triangle, then the fragment and the corresponding pixel will be black, whereas if it happens to land on a white triangle, the fragment and the corresponding pixel will be white. In the worst case, the resulting colors will form a distracting moiré pattern. When the camera moves or the checkerboard is animated, these fragments and corresponding pixels will take on changing patterns of white and black. This artifact is referred to as *flickering*. Essentially, these types of errors arise because the large amount of detail in a mathematically defined triangle can never be maintained in a discrete representation of pixels. These kinds of problems are known as *aliasing*. Multiple *anti-aliasing* techniques have been developed to counter these visual artifacts (compare anti-aliased image on right with aliased image on left in Figure 17). *Multi-sampled anti-aliasing* is an easy to enable and use technique in OpenGL. *Fast approximate anti-aliasing* is a technique developed by Nvidia and used in many games. Anti-aliasing techniques will not be covered in this document.

Top-Left Rule

In general, with point sampling, a point P^d is considered inside the triangle if Equation (6) is true, that is, if $E_i(P^d) \geq 0 \; \forall \; i \in [0, 1, 2]$. However, this condition may not always work as desired in certain cases. Consider two triangles sharing an edge with the shared edge passing through a fragment's center (see the circled fragment in Figure 18).

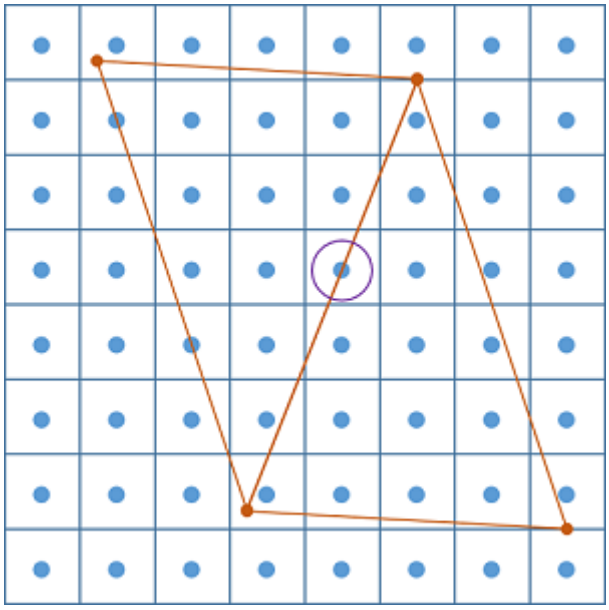

Figure 18: Fragment center on shared edge between left and right triangles

Since the circled fragment lies exactly on the shared edge, the fragment will belong to both triangles, and thus, will be visited twice by the rasterizer, thereby generating two fragments. The fragment processor will subsequently process these overlapping fragments and generate potentially different colors and alpha values for these two fragments. This gives incorrect results during shadow volume rendering and for semi-transparent surfaces. The second drawback is the worse performance caused by the generation and processing of overlapping fragments.

One approach would be to change the point-in-triangle test in Equation (6) to $E_i(P^d) > 0 \; \forall \; i \in [0, 1, 2]$. However, a fragment lying exactly on the shared edge, will now be considered to be outside both triangles. Thus, a *fragment dropout* (also known as *pixel dropout*) is generated causing an undesired crack or hole between connected triangles of an otherwise solid surface. A consistent tie-breaker rule is required to determine which triangle claims ownership of fragments whose centers are on a shared edge. The OpenGL specification only requires that if two triangles lie on either side of a common edge on which a fragment center lies, then exactly one of the triangle results in the production of the fragment during rasterization.

The tie-breaking rule used in this document is called the ***top-left rule***: a fragment center is defined to lie inside a triangle if it lies on the top edge or left edge of a triangle. Figure 19 illustrates two triangles defined by four vertices: $P_0^d = (0.5, 0.5)$, $P_1^d = (7.5, 0.5)$, $P_2^d = (7.5, 7.5)$, and $P_3^d = (0.5, 7.5)$. Assume left $\Delta P_0^d P_1^d P_3^d$ and right $\Delta P_3^d P_1^d P_2^d$ are to be rendered with red and green colors, respectively.

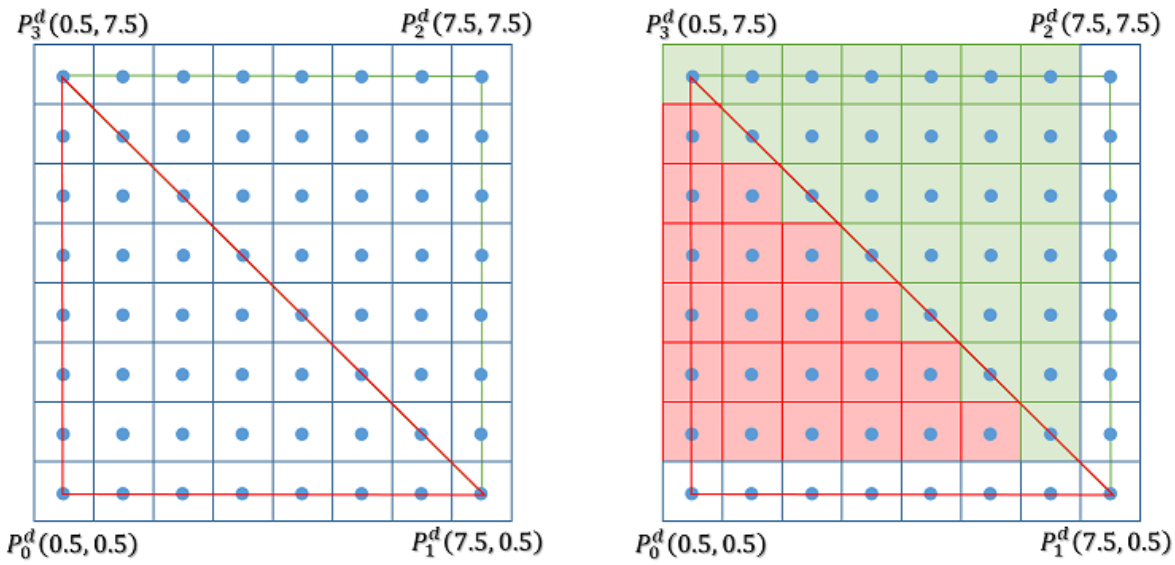

Figure 19: Illustration of the top-left rule. Left: Outline of two triangles sharing a common edge. Right: Top-left rule applied.

The top-left rule's "left" part assigns fragment centers on the shared edge to right (green) $\Delta P_3^d P_1^d P_2^d$ since they're on its left edge $\overline{P_3^d P_1^d}$ and not to $\Delta P_0^d P_1^d P_3^d$ (since they're on its right edge $\overline{P_1^d P_3^d}$). Using the same argument, fragment centers on right edge $\overline{P_1^d P_2^d}$ of $\Delta P_3^d P_1^d P_2^d$ are not inside the triangle (but are assigned to an adjacent triangle on the right that is not shown in the figure). Similarly, fragment centers on the left edge $\overline{P_3^d P_0^d}$ of $\Delta P_0^d P_1^d P_3^d$ belong to $\Delta P_0^d P_1^d P_3^d$. The "top" part of the rule assigns the fragment centers on top edge $\overline{P_2^d P_3^d}$ of $\Delta P_3^d P_1^d P_2^d$, while the fragment centers on the bottom edge $\overline{P_0^d P_1^d}$ of $\Delta P_0^d P_1^d P_3^d$ are considered outside the triangle.

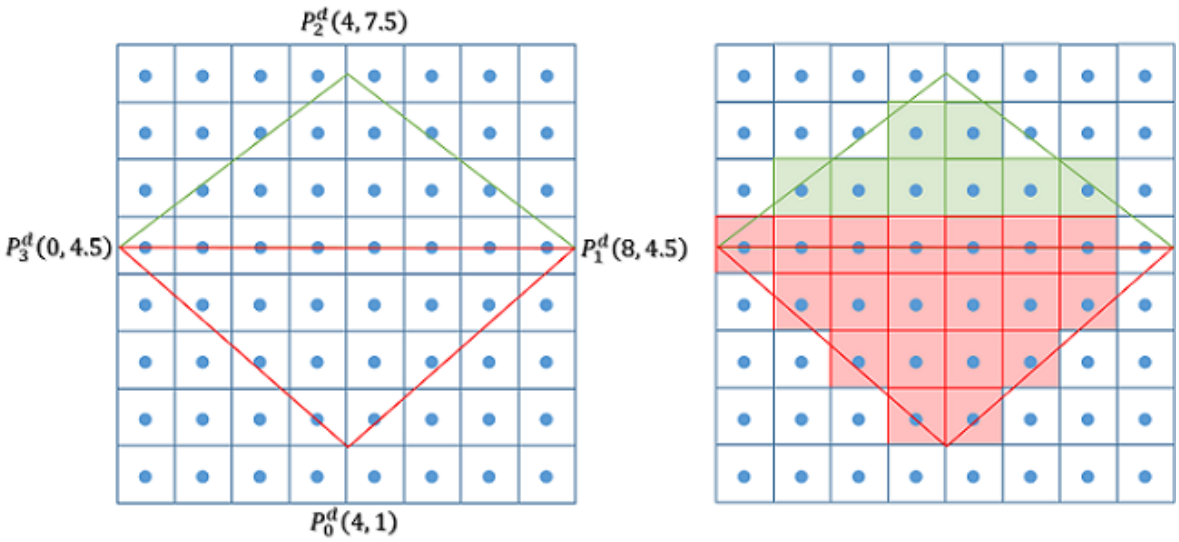

Figure 20: Illustration of the top-left rule. Left: Outline of triangles. Right: Top-left rule applied.

In Figure 20, the top-left rule's "top" part assigns fragment centers on the shared edge to bottom (red) $\Delta P_0^d P_1^d P_3^d$ since they're on its top edge $\overline{P_1^d P_3^d}$ and not to $\Delta P_3^d P_1^d P_2^d$ (since they're on its bottom edge $\overline{P_3^d P_1^d}$). More examples of the top-left rule are illustrated in the [Direct3D 11 documentation](#).

Point Inside Triangle with Top-Left Rule

A work around is required to incorporate the top-left rule for the point-in-triangle test from Equation (6) implemented as functions `PointInEdge` and `PointInTriangle`. The idea is to only include points, $P^d = (x^d, y^d)$, that are completely inside the triangle ($E_i(P^d) > 0$), and exclude points completely outside ($E_i(P^d) < 0$). If neither of these conditions are satisfied, then the point will be on a triangle edge ($E_i(P^d) = 0$). Points on the triangle's left or right edges can be identified by evaluating a_i. Assuming the convention that triangle vertices have counter-clockwise winding, left edges will have $a_i > 0$ while right edges will have $a_i < 0$. To incorporate the "left" part of the top-left rule, points on left edges ($E_i(P^d) = 0 \text{ and } a_i > 0$) will be included while points on right edges ($E_i(P^d) = 0 \text{ and } a_i < 0$) will be excluded. Next, top or bottom horizontal edges need to be considered (if an edge is not horizontal, then it is considered to be a left or right edge). A horizontal edge will have $a_i = 0$. Because of the counter-clockwise winding of triangle vertices, a horizontal edge will be a top edge if $b_i < 0$ and a bottom edge if $b_i > 0$. Because of the "top" part of the top-left rule, points on top edges are included if ($E_i(P^d) = 0 \text{ and } a_i = 0 \text{ and } b_i < 0$) and points on bottom edges are excluded if ($E_i(P^d) = 0 \text{ and } a_i = 0 \text{ and } b_i > 0$).

The `PointInEdge` is rewritten to include the work around for the top-left rule: the function will return `true` if the point is completely inside the edge or on a "top" or "left" edge, otherwise the function returns `false`:

```
1  // incorporates work around for top-left rule
2  PointInEdgeTopLeft(edge equation E, arbitrary point P) {
3    eval = E.a*P.x + E.b*P.y + E.c
4    if eval > 0 return true  // completely inside edge
5    if eval < 0 return false // completely outside edge
6    if E.a > 0 return true   // on left edge
7    if E.a < 0 return false  // on right edge
8    if E.b < 0 return true   // on top edge
9    return false             // on bottom edge
10 }
```

Paying attention to lines 6, 7, and 8 of function `PointInEdgeTopLeft`, it is observed that none of these three conditions depend on test for insideness or outsideness of arbitrary point P^d with respect to edge E_i. Instead, the conditions on lines 6, 7, and 8 involve constants a_i and b_i for edge E_i. The code can then be optimized by precomputing a boolean constant tl_i (for *top-left condition*) for each edge E_i as follows:

$$tl_i = \begin{cases} \text{boolean } (a_i > 0), \text{if } a_i \neq 0 \\ \text{boolean } (b_i < 0), \text{else} \end{cases} \tag{11}$$

where $\text{boolean } (expr)$ returns true if $expr$ evaluates true, otherwise it returns false.

First, the previous version of function `EdgeEqn` is modified to incorporate this precomputation of boolean constant tl:

```
// edge equation calculations for top-left rule
EdgeEqnTopLeft(start point P0, end point P1) {
  a = P0.y - P1.y
  b = P1.x - P0.x
  c = P0.x*P1.y - P1.x*P0.y
  // tl has boolean type
  tl = (a != 0.0) ? (a > 0.0 ? true : false) : (b < 0.0 ? true : false)
}
```

Using boolean value tl_i for each edge E_i, the `PointInEdgeTopLeft` function can be simplified to the following code:

```
// optimized work around for top-left rule
PointInEdgeTopLeftOptimized(edge equation E, arbitrary point P) {
  eval = E.a*P.x + E.b*P.y + E.c
  return (eval > 0 || (eval == 0 && E.tl == true)) ? true : false
}
```

The previous code `PointInTriangle` implementing the point-in-triangle test can be rewritten to take advantage of the optimized `PointInEdgeTopLeftOptimized` test, as shown below:

```
PointInTriangleOptimized(three edge equations E0, E1, and E2, arbitrary point P) {
  if (PointInEdgeTopLeftOptimized(E0, P) && PointInEdgeTopLeftOptimized(E1, P) &&          PointInEdgeTopLeftOptimized(E2, P)) return true
  return false
}
```

Bounding Box Triangle Traversal

Using the function `PointInTriangleOptimized`, a triangle traversal algorithm that finds the fragments inside the triangle can be formulated. A naïve method would be to test every fragment center (x^d, y^d) in the destination framebuffer grid for inclusion in the triangle by executing `PointInTriangleOptimized` for each fragment center. However, this method would require many unnecessary inclusion tests, especially for triangles with small areas. A better solution is to restrict the inclusion tests to those fragments inside the axis-aligned bounding box of the triangle. As the triangle is fully inside the bounding box, every fragment belonging to the triangle will be tested for inclusion. Once the bounding box is computed, the fragments can be visited in a left-to-right/bottom-to-top order from the bottom-left corner of the box to the top-right corner.

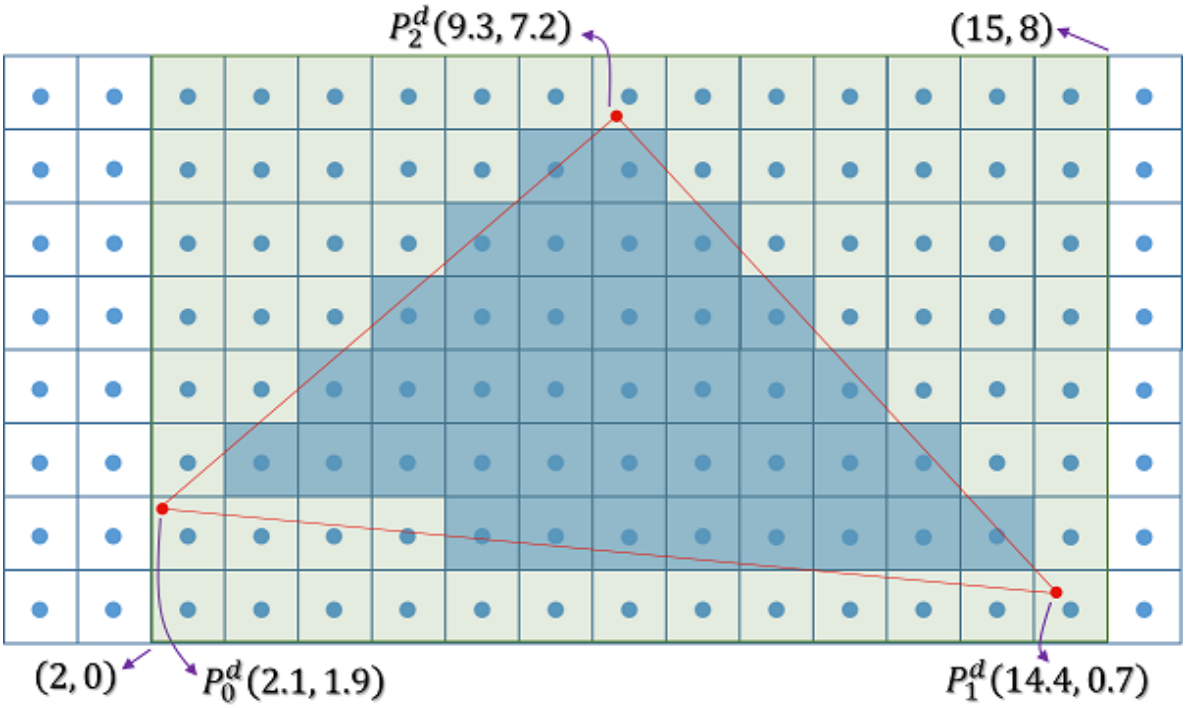

Figure 21: Bounding box triangle traversal. The light green fragments are traversed and the dark blue fragments are selected as inside the triangle.

Figure 21 illustrates a triangle with vertices $P_0^d = (2.1, 1.9)$, $P_1^d = (14.4, 0.7)$, and $P_2^d = (9.3, 7.2)$. The bounding box has a minimum-maximum representation of $(2, 0)$ and $(14, 7)$ and fragments inside the box have light green color. The dark blue fragments are actually inside the triangle. The construction of the $2D$ axis-aligned bounding box for a triangle is left to the reader as an exercise. As can be seen, the bounding box traversal scheme is not very efficient since many more fragments are visited then are actually inside the triangle. More efficient traversal schemes are described here.

Basic Triangle Rasterization Algorithm

Edge equations and the bounding-box traversal technique are brought together to implement the code for triangle rasterization:

```
TriangleRasterizer(triangle T, color rgb) {
  if (T is back-face) return
  calcuate edge equations
  calculate axis-aligned bounding box for T
  let AABB have configuration min(xmin, ymin) and max(xmax, ymax)
  for (y = ymin; y < ymax; ++y) {
    for (x = xmin; x < xmax; ++x) {
      if (PointInTriangleOptimized(edge equations,
                                   fragment position(x+0.5, y+0.5)))
        setpixel(x, y, rgb) // set framebuffer location (x, y) with color rgb
    }
  }
}
```

Incrementally Evaluating Edge Equations

Evaluating an edge equation $E(x^d, y^d) = ax^d + by^d + c$ at a fragment center (x^d, y^d) requires two multiplications and two additions. There are three edges to evaluate at potentially millions of fragment centers. An important optimization arises from the observation that edge equations are linear functions. Once a linear edge equation $E(x^d, y^d) = ax^d + by^d + c$ is evaluated at fragment center (x^d, y^d), it can be evaluated at a neighboring fragment ***incrementally***. For example, moving to fragment $(x^d + 1, y^d)$ by taking one step horizontally to the right from current fragment (x^d, y^d) means that edge equation $E(x^d + 1, y^d)$ can be computed by just adding constant a to the previous evaluation $E(x^d, y^d)$:

$$\begin{aligned} E(x^d + 1, y^d) &= a(x^d + 1) + by^d + c \\ &= ax^d + by^d + c + a \\ \implies E(x^d + 1, y^d) &= E(x^d, y^d) + a \end{aligned} \tag{12}$$

Similarly, when moving from fragment (x^d, y^d) by one step vertically to the fragment $(x^d, y^d + 1)$, the edge equation $E(x^d, y^d + 1)$ can be computed by just adding constant b to the previous evaluation $E(x^d, y^d)$:

$$\begin{aligned} E(x^d, y^d + 1) &= ax^d + b(y^d + 1) + c \\ &= (ax^d + by^d + c) + b \\ \implies E(x^d, y^d + 1) &= E(x^d, y^d) + b \end{aligned} \tag{13}$$

More generally, Equations (12) and (13), can be rewritten for general increment $\vec{t} = (t_x, t_y)$ from previous evaluated point $P^d = (x^d, y^d)$ as:

$$\begin{aligned} E(P^d + \vec{t}) &= a(x^d + t_x) + b(y^d + t_y) + c \\ &= (ax^d + by^d + c) + (at_x + bt_y) \\ \implies E(P^d + \vec{t}) &= E(P^d) + \vec{n} \bullet \vec{t} \end{aligned} \tag{14}$$

where $\vec{n} = (a, b)$ is the inward normal to the edge $E(x^d, y^d) = ax^d + by^d + c$ and $\vec{t}$ represents displacement from fragment P^d to a new fragment $P^d + \vec{t}$.

Basic Algorithm with Incremental Evaluation of Edge Equations

Enhancing the basic triangle rasterization algorithm with results from Equations (12) and (13) provide a more efficient working implementation of a triangle rasterizer:

```
TriangleRasterizer(triangle T, color rgb) {
  if (T is back-face) return

  compute edge equation E0 (a0, b0, c0, tl0) for edge from P1 to P2
  compute edge equation E1 (a1, b1, c1, tl1) for edge from P2 to P0
  compute edge equation E2 (a2, b2, c2, tl2) for edge from P0 to P1

  calculate axis-aligned bounding box for T with
  configuration min(xmin, ymin) and max(xmax, ymax)

  Eval0 <- E0(xmin+0.5, ymin+0.5)
  Eval1 <- E1(xmin+0.5, ymin+0.5)
  Eval2 <- E2(xmin+0.5, ymin+0.5)

  for (y = ymin; y < ymax; ++y) {
    HEval0, HEval1, and HEval2 are start values for horizontal span
    for (x = xmin; x < xmax; ++x) {
      if (fragment(x+0.5, y+0.5) in triangle) {
        setpixel(x, y, rgb)
      }

      incrementally update copies of HEval0, HEval1, and HEval2 using
      corresponding edge constants a0, a1, and a2, respectively for
      adjacent fragment (x+1, y)
    }

    incrementally update Eval0, Eval1, and Eval2 using
    corresponding edge constants b0, b1, and b2, respectively for
    vertically adjacent fragment (x, y+1).
    then, Eval0, Eval1, and Eval2 are edge functions to start
    evaluating horizontal span at (xmin, y+1)
  }
}
```